

Moby Dicks

We recommend books you'll actually finish.

AATISH · BRANDT · SAI

Advanced MLOps · Milestone 5 · Spring 2026

A reading curriculum, not a recommendation list.

THE PROBLEM

*Finding books to read is easy.
Finishing them is hard.*

Reddit, X, and book forums are full of people asking why they can't finish books anymore.

Goodreads, ChatGPT, and "best of" lists optimize for what you'll like — not what you'll finish.

OUR SOLUTION

Three inputs. One five-book curriculum.

Topic — what you want to read about

Time — commute, evening, weekend, long project

Difficulty — casual, moderate, challenging

Each pick comes with a written justification for why it belongs in your curriculum and where it sits in the order.



Where we stand: opportunities and risks.

STRENGTHS

- Real, common problem — high user motivation
- Team genuinely cares about reading
- Tight scope: 1 form, 1 curriculum, 5 books

WEAKNESSES

- Limited frontend experience (mitigated by Reflex)
- Cold-start: no behavioral data on day one

OPPORTUNITIES

- No popular product targets completion specifically
- LLMs return static lists with no follow-up loop
- Check-in data becomes a defensible moat

THREATS

- LLM memory features closing the gap
- Goodreads dataset access could change
- User churn if first curriculum misses

Real catalog. No hallucinated titles.

2.36M

Goodreads books in BigQuery

page counts · ratings · shelf labels
author graph · publication year · language

HOW WE USE IT

- Filter by language, page range, and rating count to drop noise
- Score candidates by keyword matches against shelf metadata
- Top 25 by relevance + popularity become Gemini's candidate pool

PLANNED ADDITIONS

- Reddit / X literature threads via PRAW (sequencing signals)
- Open Syllabus Project (university reading order)
- Project Gutenberg downloads (completion proxy for public-domain works)



Where we are, where we're going.

1

Catalog + SQL filter

Goodreads dataset loaded into BigQuery; SurveyFilters schema and shelf-keyword scoring shipped.

SHIPPED

2

Prediction pipeline

Gemini Flash extracts filters, then orders a 5-book curriculum from the candidate pool.

SHIPPED

3

Reflex frontend

Three-input form, loading state, curriculum cards, deployed via Cloud Run.

SHIPPED

4

Check-in loop

Per-book check-ins, momentum score, reminders, and Cloud Logging memory.

NEXT



Comfortably inside the \$150 GCP credit cap.

< \$0.001

per curriculum request

*Two Gemini Flash calls + one BigQuery scan,
in a 10-second end-to-end response.*

MONTHLY ESTIMATE (50 USERS · 3 CALLS EACH)

Component	Cost
Cloud Run (scale-to-zero)	\$5.00
Cloud SQL (smallest tier)	\$30.00
Operations / logging	\$5.00
Gemini Flash (150 calls)	\$0.07
Total	\$40.07

PART TWO

Final Technical Slides

System design · stack · request flow · demo · repository

The slides that follow contain the final system design and technical artifacts for Milestone 5.

What we built, in 30 seconds.

Moby Dicks is a reading-curriculum generator.

A user submits a topic, a time budget, and a difficulty level. The app returns a five-book curriculum with a written justification per book and an overall reading arc.

INPUT

One survey, three fields

Topic · time-to-read · difficulty. No 8-message ChatGPT thread.

ENGINE

Two Gemini Flash calls

Extract → fetch → curate. Grounded in a 2.36M-book BigQuery catalog.

OUTPUT

Five-book curriculum

Ordered by progressive difficulty, each pick justified, ~10 s end-to-end.



GCP services + the four required pieces.

FRONTEND + BACKEND

Reflex

Pure-Python UI, compiles to React. No JS.

DATA WAREHOUSE

BigQuery

2.36M+ Goodreads books; SQL-driven candidate pool.

LLM

Vertex AI • Gemini Flash

Two structured-JSON calls per request.

CONTAINERIZATION

Docker

Single image: FastAPI on :8000, MLflow on :5000.

CI / CD • SCHEDULING

GitHub Actions

Nightly cron + manual dispatch run pytest in Ubuntu 24.04.

EXPERIMENT TRACKING

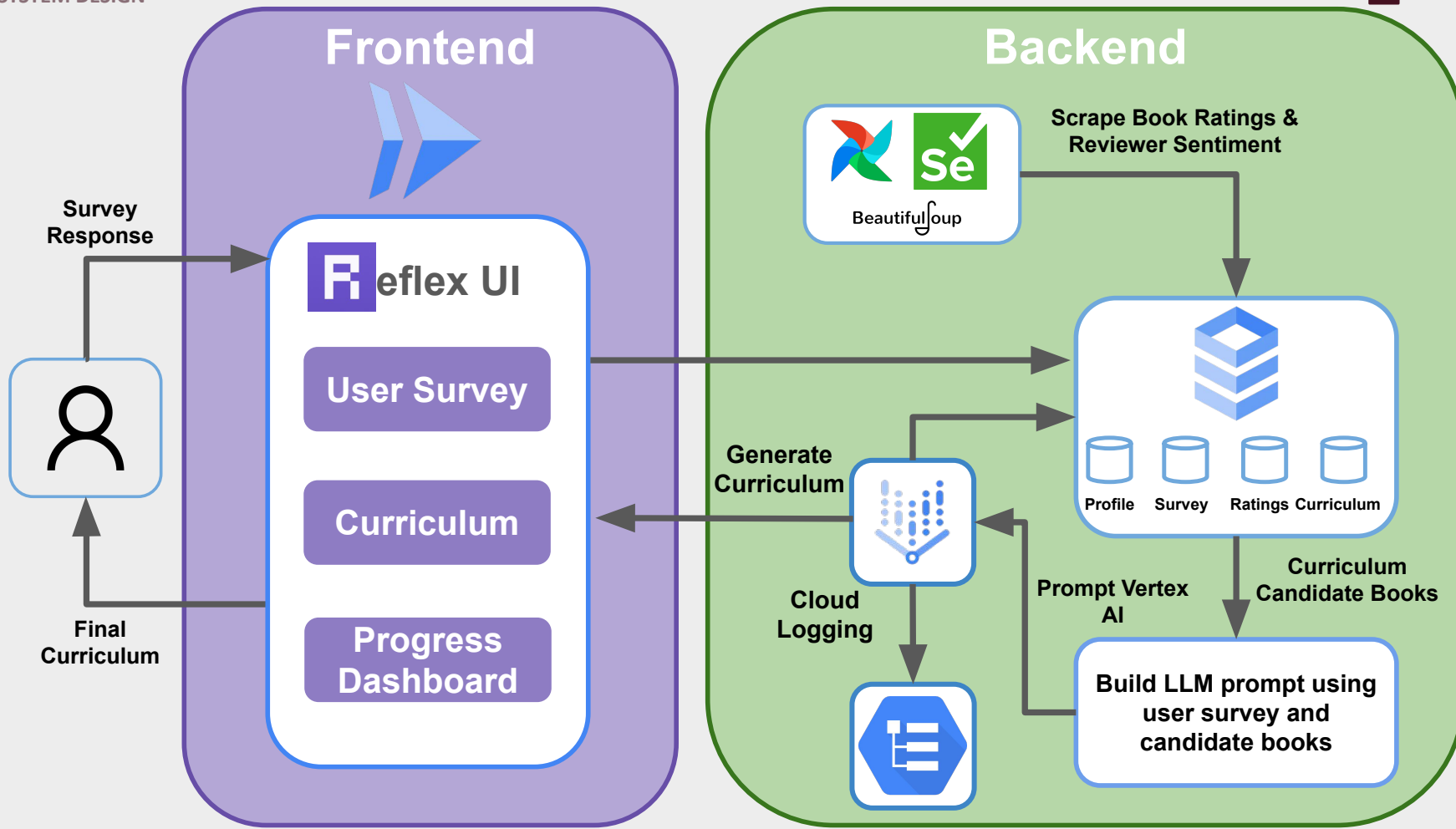
MLflow

Tracking server inside the container; runs logged per build.

HOSTING + SECRETS

Cloud Run + Secret Manager

Scale-to-zero service, IAM-authenticated, secrets injected at boot.



Goodreads catalog in BigQuery, queried at request time.

BIGQUERY

2.36M

books in the table

POPULATED FROM

*UCSD Goodreads dataset (Wan & McAuley
2018) loaded via load_to_bigquery.sh*

ROW SHAPE (per book)

book_id, title, description, num_pages, publication_year
average_rating, ratings_count, language_code
popular_shelves[] ← shelf-name + count tuples
authors[] ← author_id graph for de-dup

CANDIDATE SCAN (candidates.py)

1. Filter by language, page range, ratings_count >= 100
2. Strip noise shelves (to-read, favorites, kindle, ...)
3. Score by overlap between user keywords and shelf tokens
4. Deduplicate by work_id, return top 200, take top 25 to model



Two schema-enforced Gemini Flash calls.

CALL 1 · EXTRACT

Survey → *structured filters*

Model: *gemini-2.0-flash via Vertex AI*

Temperature: 0.2 · **Output:** JSON

Schema: *SurveyFilters (Pydantic)*

Maps free text ("evening read", "18th-century Russian lit") to keywords, page range, era, and difficulty.

Why separate: *lets us run a deterministic SQL query before the model sees any books — cheaper and grounded.*

CALL 2 · CURATE

25 candidates → *5-book curriculum*

Model: *gemini-2.0-flash via Vertex AI*

Temperature: 0.3 · **Output:** JSON

Schema: *Curriculum (week, title, author, reason, overall_arc)*

Receives top-25 candidates and orders them by progressive difficulty.

No hallucinations: *the model can only choose from real book_ids in the pool. Phase 2 will add an MLflow-tracked reranker over check-in data.*

Testing & nightly evaluation.

38

pytest cases

9

test classes

1

nightly cron · GH Actions

0

live GCP calls in CI

WHAT WE TEST

- Schema contracts (EraRange, PageRange, SurveyFilters)
- Curriculum structure & per-pick reasoning
- Pipeline failure paths (empty pool, unrecognized topic)
- FastAPI endpoint contracts
- Prompt-injection guard on extract.py

HOW IT RUNS

GitHub Actions ([.github/workflows/nightly.yml](#))

Cron: `0 6 * * *` (2 AM ET / 11 PM PT)

Manual: `workflow_dispatch` from the Actions tab

Isolation: `google.cloud` + `mlflow` stubbed via `MagicMock` — zero live cost in CI

Failure: `::error::` annotation surfaces in the PR check



Why this isn't just "ask ChatGPT."

01 One form vs eight messages

Structured survey replaces 8 back-and-forth chat turns about reading level, time, and history.

02 Grounded in a real catalog

Every recommendation maps to a real book_id in BigQuery — no hallucinated titles, no fake editions.

03 Completion, not preference

Curriculum is built for finishing. Page counts, difficulty, and time budget are hard constraints.

04 The check-in loop

Per-book check-ins reoptimize the rest of the curriculum — impossible from a single chat query.

05 Persistent reading memory

Cloud Logging stores prior outputs and feeds them back into future curricula.

06 Cost-controlled by design

Two narrowly-scoped Gemini Flash calls + caching keep us at fractions of a cent per request.



Two-minute walkthrough.

2-MIN APP DEMO



Embedded in our GitHub repo

[YouTube](#)

WHAT'S IN THE VIDEO

- Survey form: topic, time, difficulty
- Live Gemini call — extract filters
- BigQuery candidate scan
- Curriculum returned with reasons
- End-to-end in ~10 seconds

Linked from the README of the GitHub repo, as required by the milestone rubric.

CODE · TESTS · DEMO

github.com/SaiMulagan/mobydick

DEMO VIDEO

<https://youtu.be/hFVhIdpcWMs>

ARCHITECTURE

[context/ARCHITECTURE.md](#)

PIPELINE

[pipeline.py](#) · [candidates.py](#) · [curriculum.py](#)

Thank you.

THE MOBY DICKS · AATISH · BRANDT · SAI